



CS 225

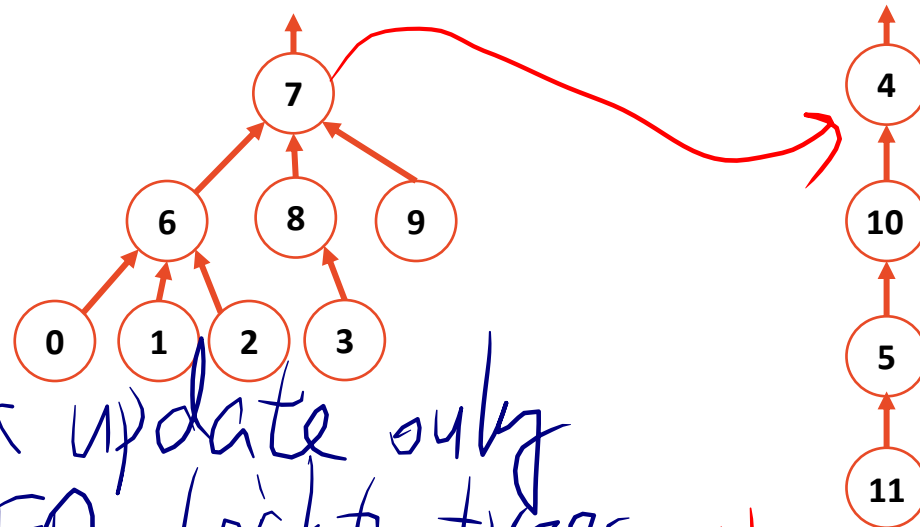
Data Structures

Volodymyr Kindratenko

Slides courtesy of Wade Fagen-Ulmschneider

Disjoint Sets – Smart Union

Store $-h-1$



12 - 0?

Height update only
w/ EQ, height trees

Union by height

-size

0	1	2	3	4	5	6	7	8	9	10	11
6	6	6	8	4	10	7	8	7	7	4	5

Idea: Keep the height of the tree as small as possible.

Union by size

0	1	2	3	4	5	6	7	8	9	10	11
6	6	6	8	4	10	7	8	7	7	4	5

Idea: Minimize the number of nodes that increase in height

Both guarantee the height of the tree is: $O(\lg n)$

Disjoint Sets Find

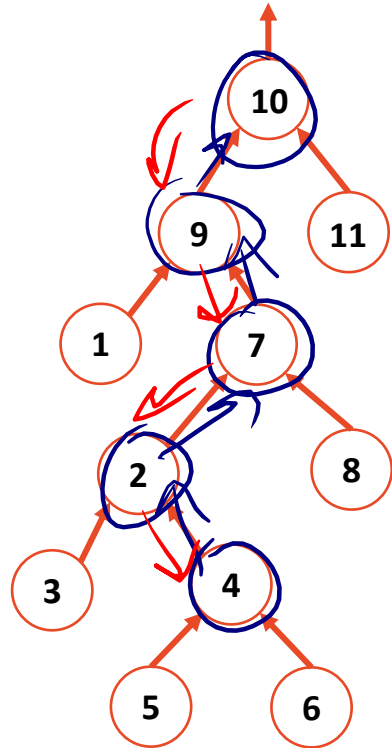
Find runs in $O(\log n)$

```
1 int DisjointSets::find(int i) {  
2     if ( s[i] < 0 ) { return i; }  
3     else { return find( s[i] ); }  
4 }
```

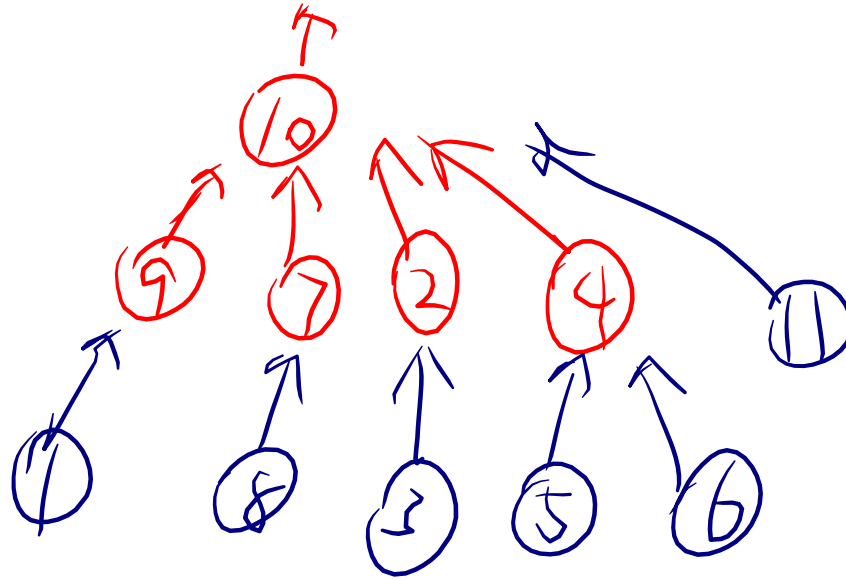
```
1 void DisjointSets::unionBySize(int root1, int root2) {  
2     int newSize = arr[root1] + arr[root2];  
3  
4     // If arr[root1] is less than (more negative), it is the larger set;  
5     // we union the smaller set, root2, with root1.  
6     if ( arr[root1] < arr[root2] ) {  
7         arr[root2] = root1;  
8         arr[root1] = newSize;  
9     }  
10  
11     // Otherwise, do the opposite:  
12     else {  
13         arr[root1] = root2;  
14         arr[root2] = newSize;  
15     }  
16 }
```

Given two roots $O(1)$

Path Compression



$\text{find}(4) = 10$



ideal $O(1)$

Disjoint Sets Analysis

The **iterated log** function:

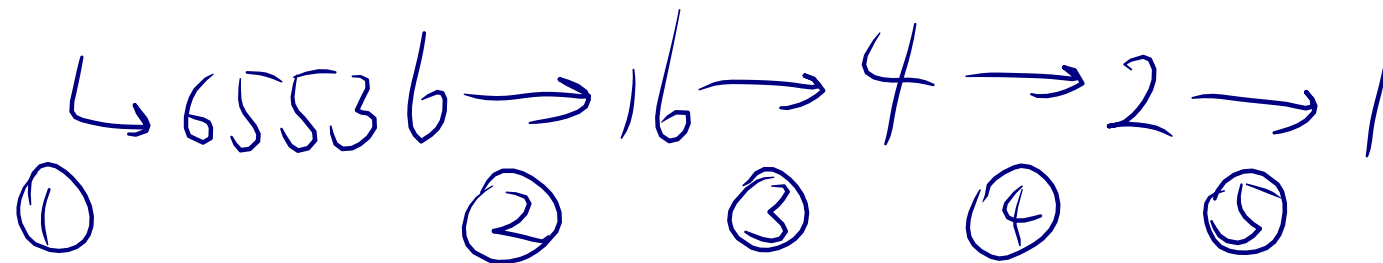
The number of times you can take a log of a number.

$$\log^*(n) =$$

$$0, n \leq 1$$

$$1 + \log^*(\log(n)), n > 1$$

What is $\log^*(2^{65536})$? $= 5$



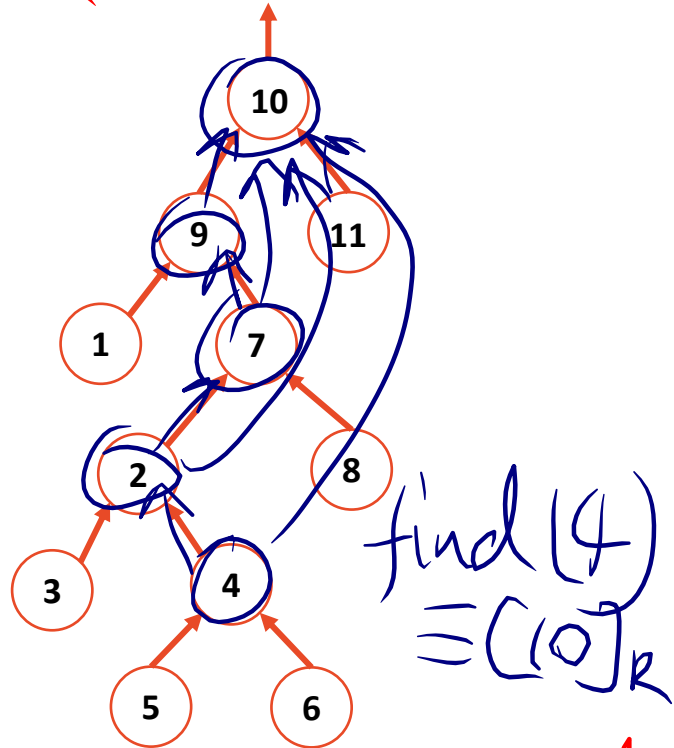
Disjoint Sets Analysis

In an Disjoint Sets implemented with smart **unions** and path compression on **find**:

Any sequence of **m union** and **find** operations result in the worse case running time of $O(\underline{m \cdot \lg^*(n)})$,
where **n** is the number of items in the Disjoint Sets.

UpTree

$[10]_R$



UpTree - Impl w/ array
- Reps. a set

A forest of up trees rep. a
collection of EQ. classes
(disjoint sets)

Near $O(1)$ to find rep. elem.

⊖ Cannot quickly find all elem. in a set

Disjoint Sets Find

```
1 int DisjointSets::find(int i) {  
2     if ( arr_[i] < 0 ) { return i; }  
3     else { return      (arr[i] = find( arr_[i] )); }  
4 }
```

```
1 void DisjointSets::unionBySize(int root1, int root2) {  
2     int newSize = arr_[root1] + arr_[root2];  
3  
4     // If arr_[root1] is less than (more negative), it is the larger set;  
5     // we union the smaller set, root2, with root1.  
6     if ( arr_[root1] < arr_[root2] ) {  
7         arr_[root2] = root1;  
8         arr_[root1] = newSize;  
9     }  
10  
11     // Otherwise, do the opposite:  
12     else {  
13         arr_[root1] = root2;  
14         arr_[root2] = newSize;  
15     }  
16 }
```

Same !

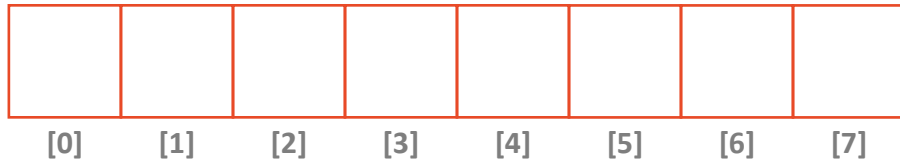
In Review: Data Structures

Array

- Sorted Array
- Unsorted Array
- Stacks
- Queues
- Hashing
- Heaps
 - Priority Queues
- UpTrees
 - Disjoint Sets

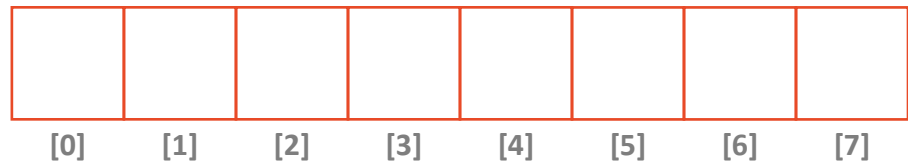
List

- Doubly Linked List
- Skip List
- Trees
 - BTree
 - Binary Tree
 - Huffman Encoding
 - kd-Tree
 - AVL Tree

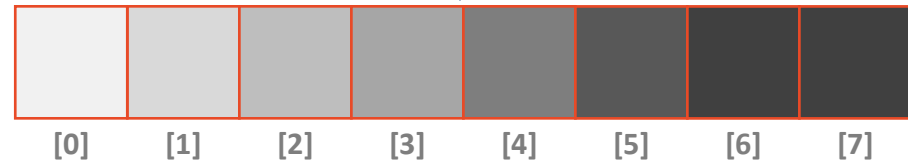


Array

-  Constant time access to any element, given an index $a[k]$ is accessed in $O(1)$ time, no matter how large the array grows
- Cache-optimized
Many modern systems cache or pre-fetch nearby memory values due the “Principle of Locality”. Therefore, arrays often perform faster than lists in identical operations.

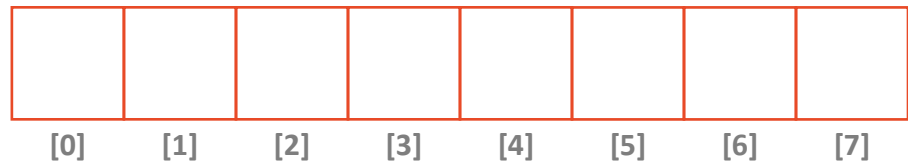


Array

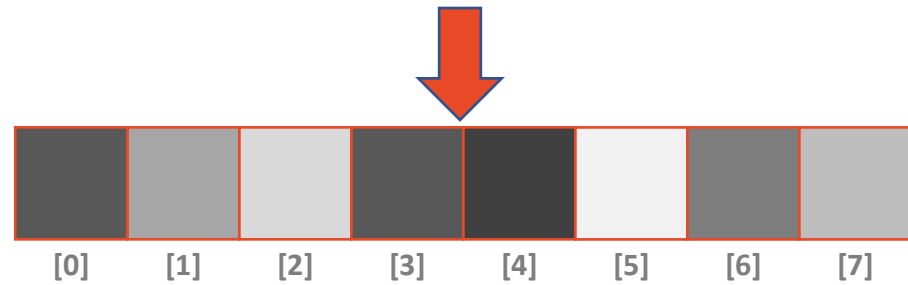


Sorted Array

- Efficient general search structure
Searches on the sort property run in $O(\log(n))$ with Binary Search
- Inefficient insert/remove
Elements must be inserted and removed at the location dictated by the sort property, resulting shifting the array in memory – an $O(n)$ operation

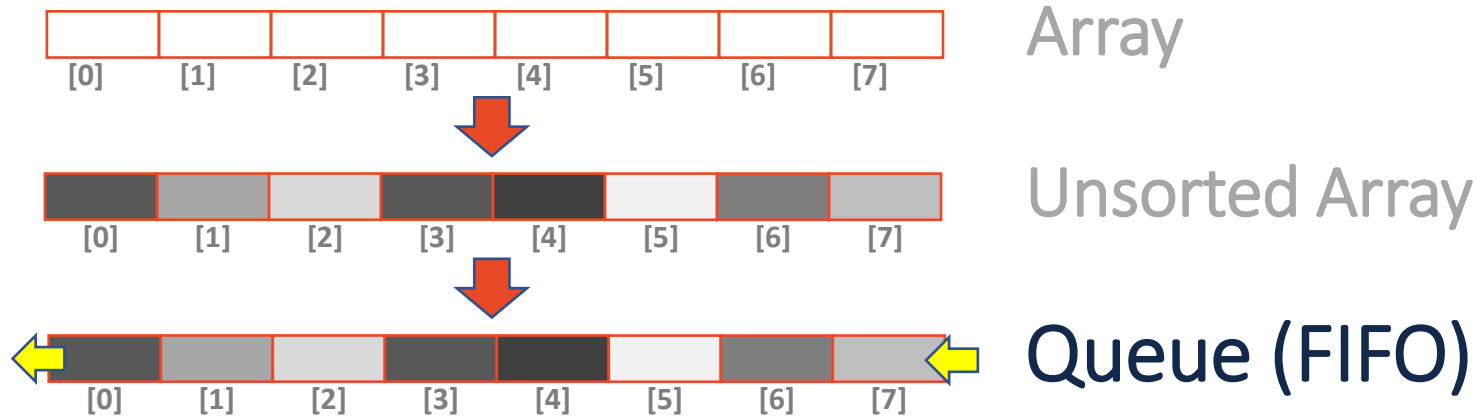


Array



Unsorted Array

- Constant time add/remove at the beginning/end
Amortized $O(1)$ insert and remove from the front and of the array
Idea: Double on resize
- Inefficient search structure
With no sort property, all searches must iterate the entire array; $O(n)$ time



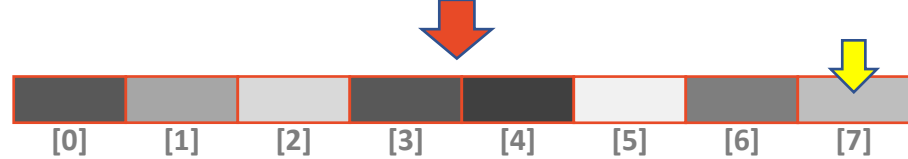
- First In First Out (FIFO) ordering of data
Maintains an arrival ordering of tasks, jobs, or data
- All ADT operations are constant time operations
enqueue() and dequeue() both run in $O(1)$ time



Array



Unsorted Array



Stack (LIFO)

- Last In First Out (LIFO) ordering of data
Maintains a “most recently added” list of data
- All ADT operations are constant time operations
`push()` and `pop()` both run in $O(1)$ time

In Review: Data Structures

Array

- Sorted Array
- Unsorted Array
- Stacks
- Queues
- Hashing
- Heaps
 - Priority Queues
- UpTrees
 - Disjoint Sets

List

- Doubly Linked List
- Skip List
- Trees
 - BTree
 - Binary Tree
 - Huffman Encoding
 - kd-Tree
 - AVL Tree

In Review: Data Structures

Array

- Sorted Array
- Unsorted Array
- Stacks
- Queues
- Hashing
- Heaps
 - Priority Queues
- UpTrees
 - Disjoint Sets

Graphs

List

- Doubly Linked List
- Skip List
- Trees
 - BTree
 - Binary Tree
 - Huffman Encoding
 - kd-Tree
 - AVL Tree